

CRYPTOGRAPHIC ARCHITECTURE WHITEPAPER

Forensic Integrity, Immutable Logs, and Legal Admissibility Compliance

Malkhana Vault employs a multi-layered cryptographic architecture to guarantee that digital evidence cannot be altered, spoofed, or deleted, satisfying **Section 63 of the Bharatiya Sakshya Adhiniyam (BSA), 2023**.

1. At-Rest Database Encryption (SQLCipher)

To protect case details and evidence logs on shared or compromised workstations:

- **Crate Linkage:** The Rust backend links `rusqlite` with the statically compiled `bundled-sqlcipher` feature, compiling SQLCipher directly into the binary.
- **Algorithm:** **AES-256-CBC** encryption is applied to every database page.
- **Key Derivation (PBKDF2):** Master passwords are run through the Password-Based Key Derivation Function 2 (PBKDF2) using **SHA-256** and **256,000 iterations**.
- **Volatile Key Storage:** The derived decryption key is stored exclusively in CPU RAM inside a locked Tauri state wrapper (`dbState`) and is never committed to persistent storage or disk swap space. Locking the vault discards the key and closes the database handle.

2. Append-Only Merkle Log Chain

Every operational event in a custody session (views, transfers, exports, edits) is chained to previous logs to prevent database tampering:

2.1 The Chain Formula

Each event hash is computed by concatenating the previous log's hash, the event classification parameters, and the active session context:

$$H_{entry} = \text{SHA-256}(H_{prev} || \text{Type}_{event} || \text{Type}_{entity} || \text{ID}_{entity} || \text{Actor} || \text{Details})$$

Where `||` represents a colon-delimited concatenation:

`prev_hash:event_type:entity_type:entity_id:actor:details`

2.2 Sealing the Session

Upon officer logout (Session Seal):

1. The backend writes a final **SESSION_CLOSED** event.
2. The final calculated entry hash becomes the **Merkle Root** representing the entire custody session.
3. This root is saved directly in the session database record. If any past row in the `session_events` table is modified, the sequential hash chain breaks, causing immediate audit verification failures.

3. §63(2)(c) System Downtime Mechanics

Under **BSA Section 63(2)(c)**, a certificate is only valid if the computer system was operating properly during the custody window, or if any downtime/interruption did not affect the integrity of the electronic record.

Malkhana Vault automates this legal attestation through the following backend workflow:

```
[System Boot] ---> Write STARTUP event to 'system_health_log'
                    |
                    +---> Checks for ungraceful SHUTDOWN
                        (e.g., POWER_LOSS, CRASH)
                    |
[Cert Generation] <-----+
                    |
                    v
Read 'system_health_log' in evidence timeline window
                    |
                    +---> No failure? ---> Generate standard §63 Certificate
                    |
                    +---> Failure detected? ---> Generate §63 Certificate with
                        Downtime Disclosure Note
```

1. **System Health Logging:** The backend writes a timestamped record to the `system_health_log` table on **STARTUP** and **SHUTDOWN** events (hooked into Tauri's lifecycle hooks). If a database is opened without a preceding graceful shutdown log, the engine logs a **POWER_LOSS** or **CRASH** event.
2. **Timeline Scan:** When the Forensic Examiner drafts a Section 63 Certificate, the `certificate_engine` queries the `system_health_log` for all events occurring between the evidence `seized_at` time and the current timestamp.
3. **Disclosure Generation:** If any system interruptions are found, the engine automatically appends a compliance note detailing the exact downtime dates, times, and restoration results, verifying that the local SQLCipher journaling (WAL mode) successfully protected the database from corruption.